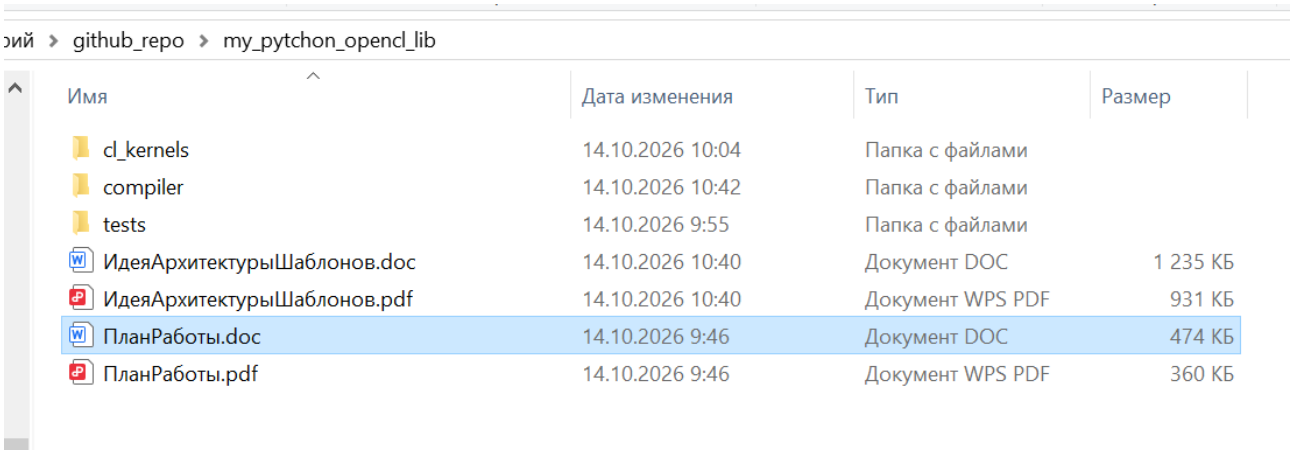
Вот текущая архитектура будущей библиотеки



Нам надо скорректировать LocalRunner таким образом,чтобы он мог корректно обрабатывать массивы с разной длиной,а также поддерживал чтение значений внешнимх переменных.
К примеру :

```
X=array(7000)
Y=array(3400)
q=0.33
LocalRunner.__call__(
        X=sinx(X)
        Y=sin(Y)
        X=X-q
        Y=y-q
)
```

Для этого нам надо передавать в ядро 1-мерный размер массивов (условно 64*64 → передаем 4096) , а вызовы функций оборачивать в условия :

```
if(tid< <variebla_name>Size){
//action
}
```

Вот текущий код:

```
    def __call__(self, lambda_func):
        try:
            raw_code = inspect.getsource(lambda_func).strip()
        except Exception:
            raise RuntimeError("LocalRunner не смог прочитать исходный код лямбды.")

        raw_code = " ".join(raw_code.split())
        match = re.search(r'(?:LocalRunner|runner)\(\s*lambda\s*:\s*(.*?)\s*\)\s*$', raw_code)
        if not match:
            match = re.search(r'lambda\s*:\s*([^)]*?)(?:\s*\)|$)', raw_code)

        expr_str = match.group(1).strip().replace('==', '=')
        expressions = [e.strip() for e in re.split(r'[;\n]', expr_str) if e.strip()]

        detected_tensors = {}
        saved_tensors_vram = {}  # Кэш клонированных буферов для текущей лямбды
        cl_body_forward = []
        cl_body_backward = []

        closure_vars = inspect.getclosurevars(lambda_func).nonlocals

        for expr in expressions:
            if '=' in expr:
                target, operation = expr.split('=', 1)
                target = target.strip()
                operation = operation.strip()

                # ОЧИСТКА: Удаляем любые висячие закрывающие скобки
                cl_op = operation.replace('nnclloc.', 'nn_')
                while cl_op.count(')') > cl_op.count('('):
                    last_bracket = cl_op.rfind(')')
                    if last_bracket != -1:
                        cl_op = cl_op[:last_bracket] + cl_op[last_bracket+1:]

                # ============================================================
                # ИЗОЛИРОВАННЫЙ БЛОК ДЛЯ DROPOUT (Перехватываем его до общего парсера)
                # ============================================================
                if "nn_dropout" in cl_op:
                    # Ищем структуру: nn_dropout(ВЫРАЖЕНИЕ, DROPOUT_P, tid, DYNAMIC_SEED)
                    drop_match = re.search(r'nn_dropout\((.*?), (.*?), tid, (.*?)\)', cl_op)
                    if drop_match:
                        inner_expr = drop_match.group(1).strip() # то, что внутри (например, X * W)
                        p_val = drop_match.group(2).strip()
                        seed_val = drop_match.group(3).strip()

                        # 1. Формируем FORWARD Си-код для Дропаута
                        # Превращаем X * W в X[tid] * W[tid]
                        tokens_inner = re.findall(r'[a-zA-Z_][a-zA-Z0-9_]*', inner_expr)
                        cl_inner_fwd = inner_expr
                        for t in tokens_inner:
                            if t not in ['DROPOUT_P', 'DYNAMIC_SEED'] and not t.isdigit():
                                detected_tensors[t] = True
```

```
                for t in tokens_inner:
                    if t not in ['DROPOUT_P', 'DYNAMIC_SEED'] and not t.isdigit():
                        detected_tensors[t] = True
                        cl_inner_fwd = re.sub(r'\b' + t + r'\b', f"{t}[tid]", cl_inner_fwd)

                detected_tensors[target] = True
                cl_body_forward.append(f"{target}[tid] = nn_dropout({cl_inner_fwd}, {p_val}, tid, {seed_val});")

                # 2. Формируем BACKWARD Си-код для Дропаута
                if self.is_training:
                    cl_body_backward.append(f"float upstream_{target} = grad_{target}[tid];")
                    cl_body_backward.append(f"    float back_drop_{target} = nn_dropout_backward(upstream_{target}, {p_val}, tid, {seed_val});")

                    # Распределяем градиент, прошедший сквозь маску дропаута, по внутреннему выражению
                    if "*" in inner_expr:
                        u, v = [item.shape_item.strip() if hasattr(item, 'shape_item') else item.strip() for item in inner_expr.split('*')]
                        detected_tensors[u] = True
                        detected_tensors[v] = True
                        cl_body_backward.append(f"    grad_{u}[tid] += back_drop_{target} * {v}[tid];")
                        cl_body_backward.append(f"    grad_{v}[tid] += back_drop_{target} * {u}[tid];")
                    elif "+" in inner_expr:
                        vars_add = [v.strip() for v in inner_expr.split('+')]
                        for v in vars_add:
                            detected_tensors[v] = True
                            cl_body_backward.append(f"    grad_{v}[tid] += back_drop_{target};")
                    else:
                        detected_tensors[inner_expr] = True
                        cl_body_backward.append(f"    grad_{inner_expr}[tid] += back_drop_{target};")

                continue # МГНОВЕННО ПЕРЕХОДИМ К СЛЕДУЮЩЕМУ ВЫРАЖЕНИЮ, ИСКЛЮЧАЯ ДВОЙНОЙ РАЗБОР!

            # ============================================================
            # СТАНДАРТНЫЙ БЛОК ДЛЯ ОСТАЛЬНЫХ ОПЕРАЦИЙ (Сигмоиды, Тангенсы, Суммы)
            # ============================================================
            tokens = re.findall(r'[a-zA-Z_][a-zA-Z0-9_]*', cl_op)

            if target in tokens and self.is_training:
                saved_name = f"saved_{target}"
                tensor_obj = closure_vars[target]
                vram_copy = cl.Buffer(self.ctx, cl.mem_flags.READ_WRITE, tensor_obj.nbytes)
                cl.enqueue_copy(self.queue, vram_copy, tensor_obj.data_buffer)
                saved_tensors_vram[saved_name] = vram_copy

            for t in tokens:
                if (t.startswith('nn_') or
                    t in ['float', 'int', 'tid', 'unsigned', 'p', 'seed', 'p_val', 'seed_val'] or
                    t.isdigit()):
                    continue
                detected_tensors[t] = True
                cl_op = re.sub(r'\b' + t + r'\b', f"{t}[tid]", cl_op)

            detected_tensors[target] = True
            cl_body_forward.append(f"{target}[tid] = {cl_op};")

            if self.is_training:
                if "nn_sigmoid" in cl_op:
```

```
                cl_body_forward.append(f"{target}[tid] = {cl_op};")

            if self.is_training:
                if "nn_sigmoid" in cl_op:
                    src_var = re.search(r'nn_sigmoid\((.*?)\)', cl_op).group(1).replace('[tid]', '')
                    use_src = f"saved_{src_var}" if f"saved_{src_var}" in saved_tensors_vram else src_var
                    cl_body_backward.append(
                        f"float upstream_{target} = grad_{target}[tid];\n"
                        f"    float back_{src_var} = nn_sigmoid_backward({use_src}[tid], upstream_{target});\n"
                        f"    grad_{src_var}[tid] = back_{src_var};"
                    )
                elif "nn_tanh" in cl_op:
                    src_var = re.search(r'nn_tanh\((.*?)\)', cl_op).group(1).replace('[tid]', '')
                    use_src = f"saved_{src_var}" if f"saved_{src_var}" in saved_tensors_vram else src_var
                    cl_body_backward.append(
                        f"float upstream_{target} = grad_{target}[tid];\n"
                        f"    float back_{src_var} = nn_tanh_backward({use_src}[tid], upstream_{target});\n"
                        f"    grad_{src_var}[tid] = back_{src_var};"
                    )
                elif "*" in operation and "nn_dropout" not in cl_op:
                    u, v = [item.strip() for item in operation.split('*')]
                    use_u = f"saved_{u}" if f"saved_{u}" in saved_tensors_vram else u
                    use_v = f"saved_{v}" if f"saved_{v}" in saved_tensors_vram else v
                    cl_body_backward.append(
                        f"float upstream_{target} = grad_{target}[tid];\n"
                        f"    float back_{u} = upstream_{target} * {use_v}[tid];\n"
                        f"    float back_{v} = upstream_{target} * {use_u}[tid];\n"
                        f"    grad_{u}[tid] += back_{u};\n"
                        f"    grad_{v}[tid] += back_{v};"
                    )
                elif "+" in operation:
                    vars_add = [v.strip() for v in operation.split('+')]
                    cl_body_backward.append(f"float upstream_{target} = grad_{target}[tid];")
                    for v in vars_add:
                        cl_body_backward.append(f"    grad_{v}[tid] += upstream_{target};")

        # Сборка аргументов ядра
        kernel_hash = abs(hash(expr_str))
        kernel_name = f"fused_forward_{kernel_hash}"

        kernel_args = []
        for tensor in detected_tensors.keys():
            kernel_args.append(f"__global float* {tensor}")
            if self.is_training:
                kernel_args.append(f"__global float* grad_{tensor}")

        for saved_name in saved_tensors_vram.keys():
            kernel_args.append(f"__global const float* {saved_name}")

        args_str = ", ".join(kernel_args)
        forward_statements = "\n    ".join(cl_body_forward)

        full_forward_code = f"""
        {BASE_CL_FUNCTIONS}
        __kernel void {kernel_name}({args_str}, const int total_elements) {{
            int tid = get_global_id(0);
```

```
        full_forward_code = f"""
        {BASE_CL_FUNCTIONS}
        __kernel void {kernel_name}({args_str}, const int total_elements) {{
            int tid = get_global_id(0);
            if (tid >= total_elements) return;
            {forward_statements}
        }}
        """
        # >>> ВОТ СЮДА ВСТАВЛЯЕМ ПЕРВЫЙ БЛОК КОДА:
        # Находим реальные значения чисел из замыкания лямбды и вжигаем их в Си-текст
        if 'DROPOUT_P' in closure_vars:
            full_forward_code = full_forward_code.replace('DROPOUT_P', f"{float(closure_vars['DROPOUT_P'])}f")
        if 'DYNAMIC_SEED' in closure_vars:
            full_forward_code = full_forward_code.replace('DYNAMIC_SEED', f"{int(closure_vars['DYNAMIC_SEED'])}")
        program = JITProgramCache.get_or_compile(self.ctx, full_forward_code)
        kernel = getattr(program, kernel_name)

        first_tensor_name = list(detected_tensors.keys())[0]
        total_elements = closure_vars[first_tensor_name].size

        # --- ФИКСАЦИЯ БУФЕРОВ ДЛЯ FORWARD ВЫЗОВА ---
        cl_buffers_forward = []
        for tensor in detected_tensors.keys():
            tensor_obj = closure_vars[tensor]
            cl_buffers_forward.append(tensor_obj.data_buffer)
            if self.is_training:
                tensor_obj.require_grad()
                cl_buffers_forward.append(tensor_obj.grad_buffer)

        for vram_buffer in saved_tensors_vram.values():
            cl_buffers_forward.append(vram_buffer)

        # Выполняем прямой проход
        kernel(self.queue, (total_elements,), None, *cl_buffers_forward, np.int32(total_elements))
        self.queue.finish()

        # --- ИЗОЛИРОВАННАЯ ФИКСАЦИЯ БУФЕРОВ ДЛЯ BACKWARD ВЫЗОВА ---
        if self.is_training and cl_body_backward:
            backward_statements = "\n    ".join(cl_body_backward)
            backward_kernel_name = f"fused_backward_{kernel_hash}"

            full_backward_code = f"""
            {BASE_CL_FUNCTIONS}
            __kernel void {backward_kernel_name}({args_str}, const int total_elements) {{
                int tid = get_global_id(0);
                if (tid >= total_elements) return;
                {backward_statements}
            }}
            """
            # >>> ВОТ СЮДА ВСТАВЛЯЕМ ВТОРОЙ БЛОК КОДА:
            # Замораживаем маску для обратного прохода по тому же seed!
            if 'DROPOUT_P' in closure_vars:
                full_backward_code = full_backward_code.replace('DROPOUT_P', f"{float(closure_vars['DROPOUT_P'])}f")
            if 'DYNAMIC_SEED' in closure_vars:
                full_backward_code = full_backward_code.replace('DYNAMIC_SEED', f"{int(closure_vars['DYNAMIC_SEED'])}")
            # ВАЖНО: Мы сохраняем в стек КОПИЮ списка буферов (cl_buffers_forward[:]),
```

```
            """
            # >>> ВОТ СЮДА ВСТАВЛЯЕМ ВТОРОЙ БЛОК КОДА:
            # Замораживаем маску для обратного прохода по тому же seed!
            if 'DROPOUT_P' in closure_vars:
                full_backward_code = full_backward_code.replace('DROPOUT_P', f"{float(closure_vars['DROPOUT_P'])}f")
            if 'DYNAMIC_SEED' in closure_vars:
                full_backward_code = full_backward_code.replace('DYNAMIC_SEED', f"{int(closure_vars['DYNAMIC_SEED'])}")
            # ВАЖНО: Мы сохраняем в стек КОПИЮ списка буферов (cl_buffers_forward[:]),
            # которая намертво зафиксировала состояние указателей и кэшей VRAM именно на ТЕКУЩИЙ шаг!
            cl_buffers_backward = list(cl_buffers_forward)

            self.backward_stack.append((full_backward_code, backward_kernel_name, cl_buffers_backward, total_elements))

    def run_backward(self):
        if not self.backward_stack:
            print("-> Стек задач Backward пуст.")
            return

        print("\n" + "#"*70)
        print("🔍 ПОШАГОВАЯ ТРАССИРОВКА ОБРАТНОГО ПРОХОДА (DEEP DETAILED LOG)")
        print("#"*70)

        from compiler.host_returner import HostReturner
        import re

        task_idx = 1
        while self.backward_stack:
            code, name, buffers, total_elements = self.backward_stack.pop()

            print(f"\n[ЗАДАЧА НАЗАД №{task_idx}]: Вызов ядра {name}")
            print("-" * 50)

            # Извлекаем сигнатуру аргументов из сгенерированного Си-кода ядра
            # Нам нужно понять, в каком порядке идут указатели на буферы
            args_match = re.search(r'__kernel void \w+\((.*?)\)', code, re.DOTALL)
            if args_match:
                # Получаем список имён переменных из Си-кода, например: ['gate_A', 'grad_gate_A', 'saved_gate_A']
                arg_declarations = args_match.group(1).split(',')
                arg_names = [decl.strip().split()[-1].replace('*', '') for decl in arg_declarations if decl.strip()]
                # Последний аргумент обычно const int total_elements, отсекаем его
                if 'total_elements' in arg_names[-1]:
                    arg_names = arg_names[:-1]
            else:
                arg_names = [f"buffer_{i}" for i in range(len(buffers))]

            # Делаем дамп содержимого ВСЕХ задействованных буферов ДО выполнения текущего ядра
            print("Содержимое буферов во VRAM ПЕРЕД вычислением текущего шага:")
            with np.printoptions(precision=4, suppress=True, edgeitems=8, linewidth=80):
                for name_buf, cl_buf in zip(arg_names, buffers):
                    # Создаем временный суррогатный объект, чтобы HostReturner смог прочитать сырой cl.Buffer
                    class TargetWrapper:
                        def __init__(self, buf, size, queue):
                            self.data_buffer = buf
                            self.shape = (size,)
```

```
                arg_declarations = args_match.group(1).split(',')
                arg_names = [decl.strip().split()[-1].replace('*', '') for decl in arg_declarations if decl.strip()]
                # Последний аргумент обычно const int total_elements, отсекаем его
                if 'total_elements' in arg_names[-1]:
                    arg_names = arg_names[:-1]
            else:
                arg_names = [f"buffer_{i}" for i in range(len(buffers))]

            # Делаем дамп содержимого ВСЕХ задействованных буферов ДО выполнения текущего ядра
            print("Содержимое буферов во VRAM ПЕРЕД вычислением текущего шага:")
            with np.printoptions(precision=4, suppress=True, edgeitems=8, linewidth=80):
                for name_buf, cl_buf in zip(arg_names, buffers):
                    # Создаем временный суррогатный объект, чтобы HostReturner смог прочитать сырой cl.Buffer
                    class TargetWrapper:
                        def __init__(self, buf, size, queue):
                            self.data_buffer = buf
                            self.shape = (size,)
                            self.dtype = np.dtype(np.float32)
                            self.queue = queue

                    wrapper = TargetWrapper(cl_buf, total_elements, self.queue)
                    cpu_data = HostReturner.to_cpu(wrapper)
                    print(f"  • {name_buf:18} -> {cpu_data}")

            # Компилируем и запускаем ядро на GPU
            program = JITProgramCache.get_or_compile(self.ctx, code)
            kernel = getattr(program, name)
            kernel(self.queue, (total_elements,), None, *buffers, np.int32(total_elements))
            self.queue.finish() # Синхронизируем, чтобы увидеть изменения

            # Делаем дамп результатов ПОСЛЕ выполнения ядра
            print("\nСодержимое буферов во VRAM ПОСЛЕ вычисления текущего шага:")
            with np.printoptions(precision=4, suppress=True, edgeitems=8, linewidth=80):
                for name_buf, cl_buf in zip(arg_names, buffers):
                    wrapper = TargetWrapper(cl_buf, total_elements, self.queue)
                    cpu_data = HostReturner.to_cpu(wrapper)
                    print(f"  • {name_buf:18} -> {cpu_data}")

            print("-" * 50)
            task_idx += 1

        print("#"*70)
        print("🔍 КОНЕЦ ТРАССИРОВКИ ОБРАТНОГО ПРОХОДА")
        print("#"*70 + "\n")
```